CÔNG NGHỆ PHẦN MỀM MỚI (MTSE431179)

Redux

THS. NGUYỄN HỮU TRUNG

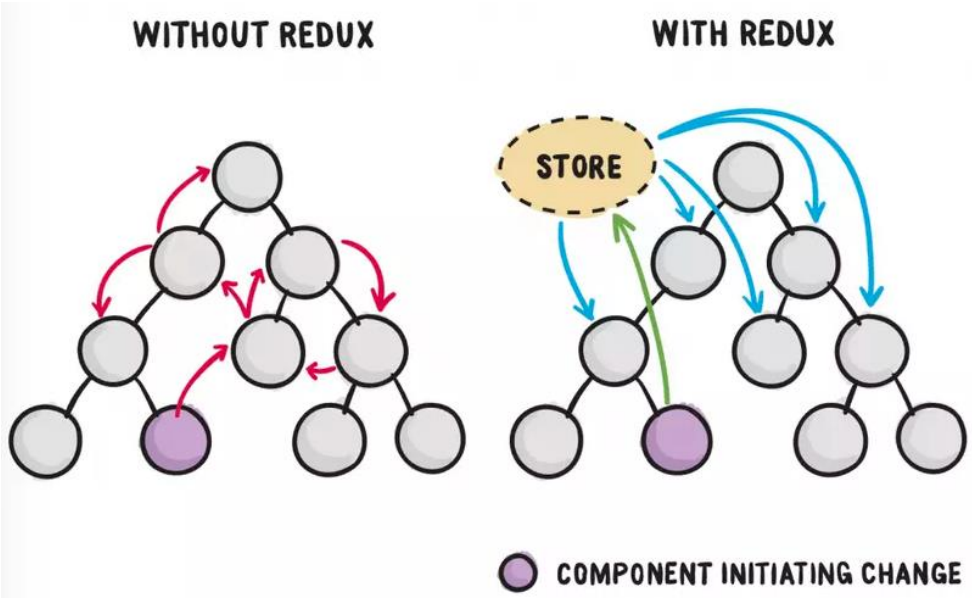
Thông tin Giảng viên

- Ths. Nguyễn Hữu Trung
- Khoa Công Nghệ Thông Tin
- Trường Đại học Sư Phạm Kỹ Thuật TP.HCM
- 090.861.7108
- trungnh@hcmute.edu.vn
- https://www.youtube.com/@baigiai

Nội dung

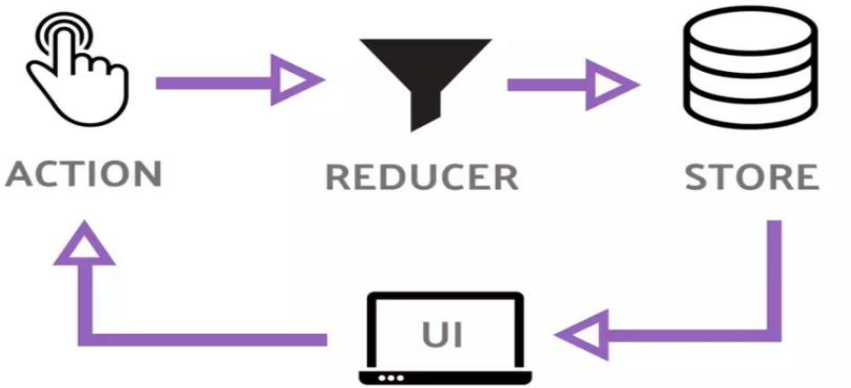
Giới thiệu về Redux

- Trước đây, muốn truyền data từ một **Component** sang **Component** khác thì tại **component** đó mình sẽ **get** nó thông qua **props**, và nếu mình gán giá trị của cho **state** của **component** là **props** mình truyền vào thì nó **render** lại **UI**. Trong trường hợp mình truyền data từ **component** ông nội đến **component** cháu thì phải thông qua **component** cha. Điều này trong các ứng dụng phức tạp sẽ rất khó để quản lý trong việc quản lý các state, chưa kể nếu có bug xảy ra thì việc debug để tìm ra vấn đề cũng không hề đơn giản, kèm theo nhiều vấn đề khác thì Redux đã xuất hiện để giải quyết vấn đề này.



Giới thiệu về Redux

- Đầu tiên redux có 3 thành phần chính:
  - action: Khi người dùng thực hiện một hành động hay một sự kiện nào đó như là submit một form hoặc call một api, thì action đó sẽ phải được gửi đến redux store bởi method store.dispatch() và mình sẽ truyền action vào. ví dụ

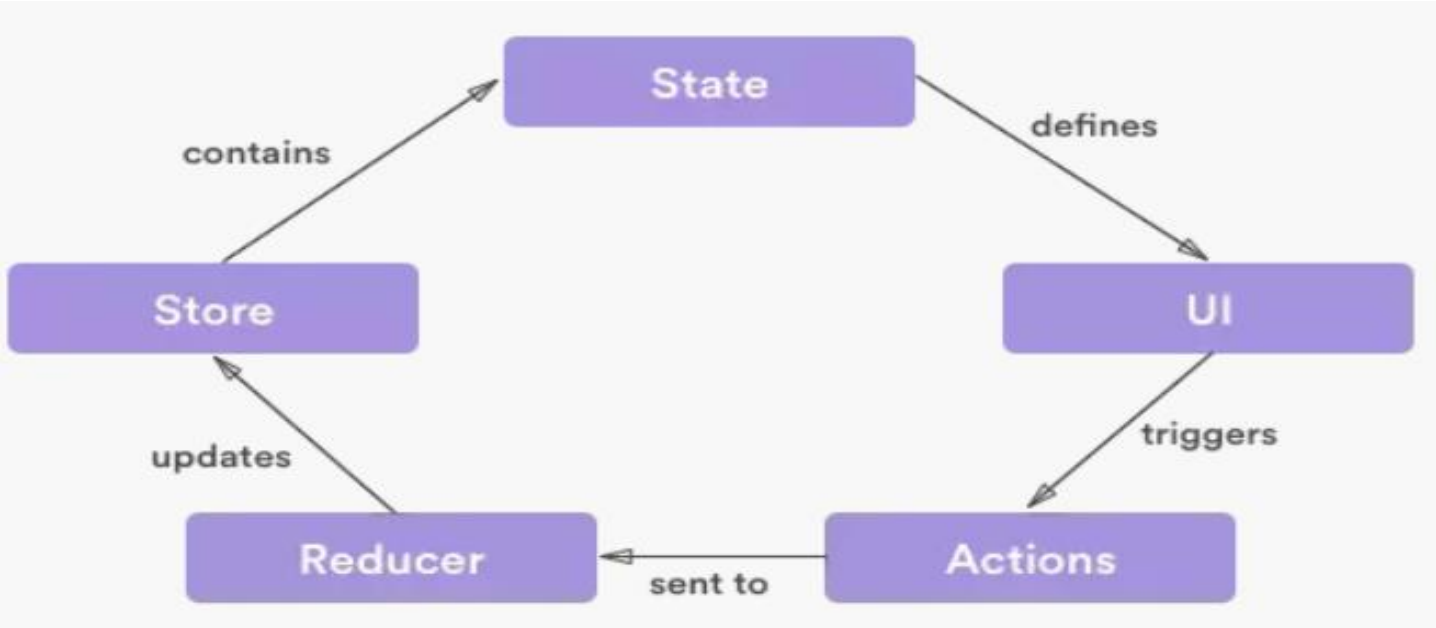



```
export const listAll = () => {
    return {
        type: Types.LIST_ALL
    };
}
store.dispatch(listAll());
```

  - Reducer: Reducer sẽ lấy các action, sau đó phân tích và trả về các state mới cho store.
  - Store: Là thành phần rất quan trọng, nó sẽ chứa các state của các component. Thông qua đó từ một component mình sẽ kết nối với store để lấy, cập nhật các state cho các component. Store luôn là nguồn dữ liệu đúng và tin cậy duy nhất.

Redux data follow

1. Khi có 1 sự kiện (event) thì sẽ phát sinh ra 1 action mô tả những gì đang xảy ra.
2. Action sẽ thực hiện điều phối Reducer xử lý event.
3. Reducer dự vào những mô tả của Action để biết cần thực hiện thay đổi gì trên State và thực hiện update.
4. Khi State được update thì tất nhiên rồi. các trigger đang theo dõi state đó sẽ nhận được thông tin update.



Middleware library

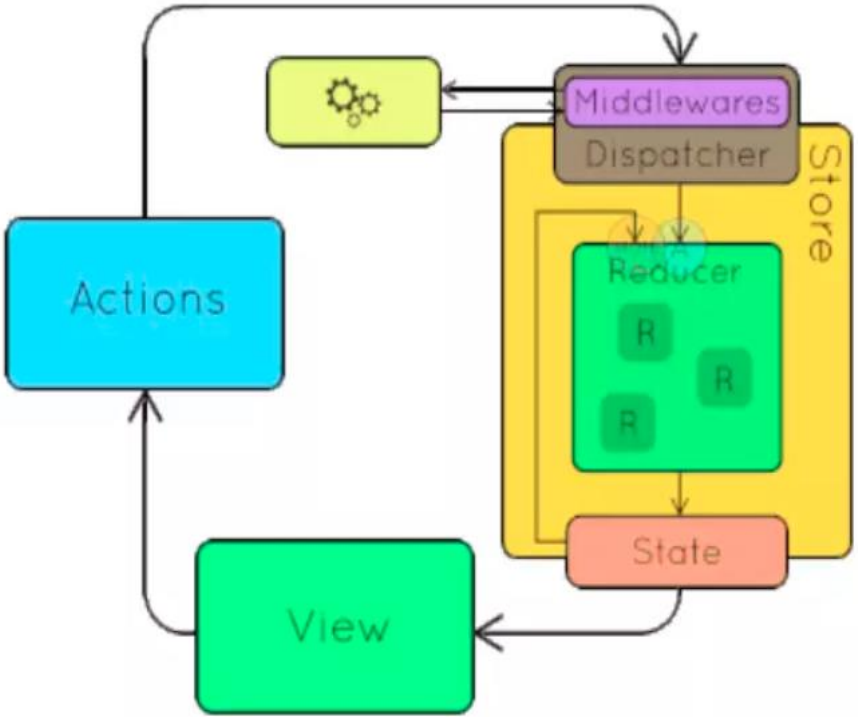
- Việc thao tác với các state ở redux store có nhiều vấn đề khác, ví dụ mình muốn gọi api, hoặc sử dụng các hàm setTimeout để thao tác các state thì sao, tất cả những việc đó được gọi chung là side effects, vậy làm thế nào để mình handle được những side effect đó.
- Thì lúc này middleware library sẽ giúp xử lý những vấn đề này. Các bạn có thể hiểu đơn gian middleware library là thành phần đứng giữa các action và reducer.
- Khi một action được dispatch vào reducer, thì nó sẽ kiểm tra xem action đó có thực thiện bất đồng bộ hay không, nếu có nó sẽ chờ cho action bất đồng bộ thực hiện xong rồi mới đưa action vào trong reducer.
- Một số library middleware thường sử dụng:
  - Redux-thunk
  - Redux-saga
  - Redux-promise

Redux-thunk

- Chắc các bạn cũng biết là action thường trả về dạng object, người ta hay gọi là **plain Javascript object**. Trong trường hợp mình muốn gọi một api để trả về một list trending thì action của mình không thể trả về một plain Javascript object thông thường được, mà mình sẽ phải trả về một function, action như vậy được gọi là async action.

```
export const fetchTrendingRequest = () => {
  return (dispatch) => {
    callAPI().then(({data})=>{
      dispatch(fetchTrending(data.data));
    })
  }
}

export const fetchTrending = (trendings) => {
  return {
    type: Types.FETCH_TRENDING,
    trendings
  }
}
```



Redux-thunk

- Chắc các bạn cũng biết là action thường trả về dạng object, người ta hay gọi là **plain Javascript object**. Trong trường hợp mình muốn gọi một api để trả về một list trending thì action của mình không thể trả về một plain Javascript object thông thường được, mà mình sẽ phải trả về một function, action như vậy được gọi là async action.

```
export const fetchTrendingRequest = () => {
  return (dispatch) => {
    callAPI().then(({data})=>{
      dispatch(fetchTrending(data.data));
    })
  }
}

export const fetchTrending = (trendings) => {
  return {
    type: Types.FETCH_TRENDING,
    trendings
  }
}
```

- Action đầu tiên của mình, ở đây mình trả về một function và function này sẽ tiến hành call api để lấy về listTrending, lúc này redux thunk nó sẽ cho phép chương trình dừng lại cho đến khi api gọi xong và trả về kết quả.
- Tiếp đến mình gọi đến một action bên dưới truyền data vừa mới get được vào và lúc này redux thunk nó sẽ kiểm tra action này không thực hiện async nên nó sẽ đưa đến cho reducer để xử lý

https://github.com/reduxjs/redux-thunk

Redux saga

- Redux saga cung cấp các hàm helper effect, các hàm này sẽ trả về một effect object chứa đựng thông tin đặc biệt chỉ dẫn middeware của Redux có thể thực hiện tiếp các hành động khác. Các hàm helper effect sẽ được thực thi trong các generator function. Generator function là một tính năng mới trong ES6, nó cũng là một function. Tuy nhiên điểm đặc biệt của function này là có thể tạm dừng để thực thi một việc khác, hoặc có thể gọi đến một Generator function khác.
- Một số helper của generator function được redux saga sử dụng:
  - **takeEvery()** : thực thi và trả lại kết quả của mọi actions được gọi.
  - **takeLastest()** : có nghĩa là nếu chúng ta thực hiện một loạt các actions, nó sẽ chỉ thực thi và trả lại kết quả của của actions cuối cùng.
  - **take()** : tạm dừng cho đến khi nhận được action
  - **put()** : dispatch một action.
  - **call():** gọi function. Nếu nó return về một promise, tạm dừng saga cho đến khi promise được giải quyết.
  - **race()** : chạy nhiều effect đồng thời, sau đó hủy tất cả nếu một trong số đó kết thúc

Redux saga & Redux Thunk

	Redux Thunk	Rudux Saga
Ưu điểm	Đơn giản, mạnh mẽ, dễ sử dụng , dễ tiếp cận đối với các bạn là mới học React	Đối với những dự án phức tạp sử dụng redux-saga code sẽ clean và dễ test hơn so với redux-thunk, giải quyết được những vấn đề về chains of promises
Nhược điểm	Chỉ phù hợp với các dự án nhỏ, xử lý logic đơn giản. Còn đối với những dự án phức tạp sử dụng redux-thunk sẽ phải tốn nhiều dòng code và gây khó khăn cho việc test các action	Phức tạp, tốn thời gian cho member mới vào team, nặng về xử lý logic, không dành cho những ứng dụng đơn giản